Projeto e Análise de Algoritmos — Lista de Exercícios III

Instituto Federal de Minas Gerais, Campus Formiga · Bacharelado em Ciência da Computação · 2026/2
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

> **Lista revisada pelo professor** (arquivo de 09/09/2026; o cabeçalho da lista manteve a data original). Entraram duas questões novas — a Questão 3 (moeda falsa) e a Questão 5 (passo indutivo sem base) — e a antiga Questão 3 passou a ser a Questão 4, sem mudança de conteúdo. A versão anterior está arquivada em `listas-de-exercicios/versoes-anteriores/`.

> **Estrutura das provas por indução.** Cada item da Questão 4 segue o mesmo esqueleto: enunciar *P(n)*; verificar a **base** no valor inicial que o enunciado exige, com os dois lados avaliados; escrever a **hipótese indutiva**; e provar o **passo**, marcando o ponto exato em que a hipótese é aplicada. Nas recorrências, o passo parte da *definição recursiva* e chega à fórmula fechada — partir da fórmula fechada querendo chegar nela mesma seria circular. A Questão 3 aplica o mesmo esqueleto a um *procedimento* (a hipótese é aplicada a um subconjunto das moedas), e a Questão 5 mostra o que sobra quando só o passo vale e a base falha.

> Cabeçalho e instruções da lista (transcrição literal)
>
> Instituto Federal de Educação, Ciência e Tecnologia · Curso: Bacharelado em Ciência da Computação · Disciplina: Projeto e Análise de Algoritmos · Professor: Mário Luiz Rodrigues Oliveira · Atividade: Lista Exercícios III · Formiga,MG, 23 de agosto de 2026
>
> **Instruções:**
>
> 1. Essa lista não será avaliada e, portanto, não precisa ser entregue. Resolva os exercícios como forma de fixação do conteúdo ministrado.
> 2. Caso você ache que falta algum detalhe nas especificações, você deverá fazer as suposições que julgar necessárias e escrevê-las junto à resolução do trabalho. Pode acontecer também que a descrição dessa atividade contenha dados e/ou especificações supérfluas para sua solução. Utilize sua capacidade de julgamento para separar o supérfluo do necessário.
> 3. Não basta escrever apenas o resultado. É necessário mostrar a resolução ou o raciocínio utilizado para responder às questões. Portanto, justifique todas as suas respostas.

Questão 1

> Enunciado
>
> Alice e Bob estão discutindo sobre o desempenho de seus algoritmos de ordenação. Alice afirma que o seu algoritmo de ordenação executando em tempo $\theta(n * \lg n)$ é melhor que algoritmo de Bob que executa em tempo $\theta(n^2)$. Para decidir qual é melhor algoritmo, implementaram e executaram ambos os algoritmos no mesmo ambiente (mesma arquitetura, mesmo sistema operacional, mesma linguagem de programação e mesmo compilador) em uma série de dados. Para espanto de Alice, descobriram que quando $n < 100$ o algoritmo $\theta(n^2)$ é mais rápido e apenas quando $n \geq 100$ é que o algoritmo $\theta(n * \lg n)$ é melhor. Explique como essa situação pode ocorrer.

Não há contradição alguma. Alice está certa *assintoticamente*, e a medição também está certa *na faixa testada*. As duas coisas convivem porque Θ descarta exatamente aquilo que decide o desempenho para n pequeno.

1. O que Θ afirma — e o que não afirma

Por definição, $T(n) = \Theta(g(n))$ significa que existem $c_1, c_2 > 0$ e $n_0 > 0$ com $0 \leq c_1 g(n) \leq T(n) \leq c_2 g(n)$ para todo $n \geq n_0$. Disso decorrem três coisas:

(i) as constantes c_1, c_2 são arbitrárias — nada limita seu tamanho; (ii) termos de menor ordem desaparecem, pois $5n \lg n + 500n + 20000$ também é $\Theta(n \lg n)$; e (iii) a garantia só vale a partir de um n_0, que pode ser grande.

Ou seja, "Θ(n lg n) é melhor que Θ(n²)" é afirmação sobre o *limite*, não sobre cada valor de n.

2. Um exemplo concreto que reproduz o cenário

Tome

$$T_A(n) = 15{\cdot}n{\cdot}\lg n + 20 \ \text{(Alice)} \quad \text{e} \quad T_B(n) = n^2 \ \text{(Bob)}.$$

T_A é $\Theta(n \lg n)$ e T_B é $\Theta(n^2)$, exatamente como Alice e Bob anunciaram. A única diferença estrutural é a constante 15 e o termo constante 20 — precisamente o que Θ apaga. O 20 tem leitura natural: é o custo fixo de preparação, como alocar o vetor auxiliar de um mergesort.

n	$T_A = 15n \lg n + 20$	$T_B = n^2$	mais rápido
1	20	1	Bob
10	518	100	Bob
50	4 253	2 500	Bob
99	9 865	9 801	Bob
100	9 986	10 000	**Alice**
200	22 952	40 000	**Alice**
1000	149 507	1 000 000	**Alice**

> O termo constante 20 não é decoração: sem ele, com $T_A(n) = 15n \lg n$, teríamos $T_A(1) = 15{\cdot}1{\cdot}\lg 1 = 0$, porque $\lg 1 = 0$. Alice "venceria" em $n = 1$ com tempo zero — um artefato degenerado que contradiz o enunciado, já que ele afirma que Bob é mais rápido para *todo* $n < 100$. Com o termo constante, Bob vence em todos os inteiros de 1 a 99, sem furos.

3. O ponto de cruzamento, algebricamente

Ignorando o termo constante para achar a ordem de grandeza do cruzamento, resolve-se

$$15{\cdot}n{\cdot}\lg n = n^2 \iff 15{\cdot}\lg n = n \iff n \,/\, \lg n = 15,$$

onde a divisão por $\lg n$ exige $\lg n > 0$, isto é, $n \geq 2$ — restrição que precisa ser declarada.

Seja $\varphi(n) = n \,/\, \lg n$. Derivando, $\varphi'(n) = \ln 2{\cdot}(\ln n - 1)/(\ln n)^2$, que é positiva quando $\ln n > 1$, ou seja $n > e \approx 2{,}718$. Logo φ é estritamente crescente **a partir de n = 3**, e só nessa região a equação $\varphi(n) = 15$ tem solução única.

$$\varphi(99) = 14{,}93 < 15 < 15{,}05 = \varphi(100).$$

> Esse *bracket* localiza a raiz da equação **simplificada** $15n{\cdot}\lg n = n^2$, que é $n \approx 99{,}563$. A raiz da equação **completa** $15n{\cdot}\lg n + 20 = n^2$ é outra: $n \approx 99{,}819$. O bracket correspondente é $T_A(99) = 9\,864{,}6 > 9\,801 = T_B(99)$ e $T_A(100) = 9\,985{,}8 < 10\,000 = T_B(100)$.

Resolvendo numericamente $T_A(n) = T_B(n)$, o cruzamento cai em $n \approx 99{,}82$ — entre 99 e 100. Portanto o **primeiro inteiro** em que Alice vence é $n = 100$, exatamente o limiar observado.

Por que não volta a inverter. Seja $h(n) = T_B(n) - T_A(n) = n^2 - 15n{\cdot}\lg n - 20$. Sua derivada é $h'(n) = 2n - 15{\cdot}\lg n - 15/\ln 2$, que é negativa só para $n \leq 53$ e positiva a partir de $n = 54$. Logo h é crescente em todo $n \geq 54$ e, como $h(99) < 0 < h(100)$, ela troca de sinal **uma única vez**. Assim Bob vence exatamente em $1 \leq n \leq 99$, sem furos, e Alice de $n = 100$ em diante, sem reversão.

> **Cuidado com dois limiares diferentes.** O número 100 é o *ponto de cruzamento entre* T_A *e* T_B, causado pela constante 15. Ele **não é o n_0 mínimo** de nenhuma das duas declarações Θ — nelas basta $n_0 = 2$ — e nada na definição de Θ o determina. (Rigorosamente, n_0 não é único: qualquer valor maior também serve, inclusive 100. O que se quer dizer é que o cruzamento não é o menor n_0 possível, e que ele vem da constante 15, não da definição.) Chamar o cruzamento de "n_0 do par" confunde duas constantes que não têm relação: uma liga um algoritmo à sua própria classe, a outra compara dois algoritmos.

4. Outros fatores que contribuem

Termos de menor ordem. O custo real de Alice poderia ser $5n \lg n + 500n + 3000$, que continua sendo $\Theta(n \lg n)$. Para n pequeno o $500n$ e a constante dominam; para n grande somem diante de $5n \lg n$.

Fatores que o modelo RAM não enxerga. O modelo cobra custo unitário por operação, mas na máquina real o algoritmo $\Theta(n^2)$ pode ter localidade de referência muito melhor (varre vetores contíguos, que cabem em cache), pode ser iterativo — sem o custo de empilhar chamadas recursivas — e pode operar *in place*, sem alocar memória auxiliar nem pagar cópias. Todos esses custos entram na constante multiplicativa do algoritmo assintoticamente melhor.

É exatamente o caso do *insertion sort* ($\Theta(n^2)$ no pior caso) contra o *mergesort* ($\Theta(n \lg n)$): para entradas pequenas o insertion sort é mais rápido na prática, e por isso implementações reais de mergesort e quicksort trocam para insertion sort quando o subproblema fica abaixo de um limiar de algumas dezenas de elementos.

5. Conclusão

A afirmação de Alice é sobre o comportamento quando $n \to \infty$, e é verdadeira: a partir de $n = 100$ seu algoritmo é mais rápido, e a vantagem cresce sem limite, pois $T_B(n)/T_A(n) \approx n/(15 \lg n) \to \infty$. A observação empírica apenas revela onde está o ponto de cruzamento desse par de implementações — e quem o determina são as constantes, que a notação Θ deliberadamente ignora.

Questão 2

> Enunciado
>
> Considere
>
> $$f(n) = \{\ n \quad \text{se } n \text{ é ímpar};\quad n^2 \quad \text{se } n \text{ é par}\ \}$$
>
> Apresente um limite assintótico para a função f tão firme (restrito/justo) quanto possível, usando a notação Big O.

Resposta: $f(n) = O(n^2)$, com testemunhas $c = 1$ e $n_0 = 1$.

n	1	2	3	4	5	6	7	8	9	10
f(n)	1	4	3	16	5	36	7	64	9	100

(1) Limite superior: $f(n) = O(n^2)$

Testemunhas $c = 1$ e $n_0 = 1$, fixadas antes e independentes de n. Seja $n \geq 1$:

Caso n par: $f(n) = n^2 \leq 1 \cdot n^2$, com igualdade.
Caso n ímpar: $f(n) = n$. De $1 \leq n$, multiplicando pelo positivo n, vem $n \leq n^2$. Logo $f(n) \leq 1 \cdot n^2$.

Em ambos $f(n) \geq 0$. Portanto $0 \leq f(n) \leq 1 \cdot n^2$ para todo $n \geq 1$. ∎

(2) Esse teto não pode ser trocado por um menor: f(n) não é O(n)

Nos índices *pares* a função vale exatamente n^2, então nenhuma função g com $g(n) = o(n^2)$ pode servir de teto.

Suponha por absurdo $f(n) = O(n)$: existem $c > 0$ e n_0 com $f(n) \leq c \cdot n$ para $n \geq n_0$. Escolha um n **par** com $n > \max(n_0, c)$ — existe, pois há infinitos pares. Aí $f(n) = n^2$, e a desigualdade daria $n^2 \leq c \cdot n$; dividindo por $n > 0$, $n \leq c$, contradizendo $n > c$. ∎

O mesmo argumento vale para qualquer g com $g(n) = o(n^2)$. Mais que isso: entre as funções não decrescentes — como todas as da hierarquia do anexo —, n^2 é o menor teto a menos de constante. Se g é não decrescente e $f(n) \leq c \cdot g(n)$ para $n \geq n_0$, então nos n pares $g(n) \geq n^2/c$, e nos ímpares com $n - 1 \geq n_0$ vale $g(n) \geq g(n-1) \geq (n-1)^2/c \geq n^2/(4c)$; logo $g = \Omega(n^2)$. (Sem a restrição, a própria f seria um teto mais firme: $f = O(f)$, mas n^2 não é $O(f)$.) Nesse sentido, n^2 é o teto justo.

(3) Ponto central: f(n) *não* é Θ(n²)

Θ exige teto **e** piso da mesma ordem. O teto vale, mas o piso não — e é isso que a questão testa.

Suponha por absurdo $f(n) = \Omega(n^2)$: existem $c > 0$ e n_0 com $c \cdot n^2 \leq f(n)$ para $n \geq n_0$. Escolha um n **ímpar** com $n > \max(n_0, 1/c)$. Aí $f(n) = n$, e a desigualdade daria $c \cdot n^2 \leq n$; dividindo por $n > 0$, $c \cdot n \leq 1$, isto é $n \leq 1/c$ — contradizendo $n > 1/c$. ∎

> Note que a constante c não salva a situação por menor que seja: ela apenas *afasta* o primeiro ponto de falha, sem eliminá-lo. Para $c = 1$ a desigualdade falha já em $n = 3$; para $c = 0{,}01$ falha a partir de $n = 101$; para $c = 0{,}0001$, a partir de $n = 10\,001$. Sempre restam infinitos ímpares onde falha.

Como $f(n) \geq n$ para todo $n \geq 1$, vale $f(n) = \Omega(n)$. Assim $n \leq f(n) \leq n^2$: a função fica espremida entre as duas ordens **sem ser Θ de nenhuma das duas**, porque oscila entre elas conforme a paridade.

Questão 3

> Enunciado
>
> Têm-se 2^n moedas de ouro, sendo uma delas falsa, com peso menor do que as demais. Dispõe-se de uma balança de dois pratos, sem nenhum peso. Vamos mostrar, por indução sobre n, que é possível achar a moeda falsa com n pesagens.

Bissecção: divide-se em duas metades, pesa-se uma contra a outra, e a metade mais leve é um problema do mesmo tipo com a metade das moedas. A indução formaliza isso — e o ponto delicado é enunciar $P(n)$ para *qualquer* conjunto de moedas, porque a hipótese será aplicada a um subconjunto.

1. Suposições e a proposição

Suposições declaradas. (1) Todas as moedas verdadeiras têm o mesmo peso p; a falsa tem peso $q < p$. ("As demais" são as verdadeiras; o enunciado diz que a falsa pesa menos que elas.) (2) Uma pesagem põe um conjunto de moedas em cada prato e observa um de três resultados: prato esquerdo mais pesado, prato direito mais pesado, ou equilíbrio. (3) "Com n pesagens" é lido como "com no máximo n pesagens"; o procedimento abaixo usa exatamente n. (4) n é inteiro, $n \geq 1$ (se $0 \in \mathbb{N}$, o caso $n = 0$ também vale e é tratado à parte).

Proposição. $P(n)$: para todo conjunto M de 2^n moedas no qual exatamente uma é falsa, de peso q, e as demais têm todas o mesmo peso $p > q$, é possível identificar a moeda falsa de M usando n pesagens.

> A quantificação "para todo conjunto M" é essencial: no passo, a hipótese de indução será aplicada a um **subconjunto** das moedas originais, não ao conjunto original.

2. A indução

Base (n = 1). M tem $2^1 = 2$ moedas. Põe-se uma em cada prato (1 pesagem). Um prato tem peso q e o outro, p; como $q < p$, a balança não equilibra, e a moeda do prato mais leve é a falsa. *P(1)* vale, com 1 pesagem. (Caso $n = 0$, se $0 \in \mathbb{N}$: M tem $2^0 = 1$ moeda, que é a falsa; 0 pesagens. *P(0)* vale.)

Hipótese de indução. Seja $n \geq 1$ fixado e suponha *P(n)*: em qualquer conjunto de 2^n moedas com exatamente uma falsa, mais leve, e as demais de mesmo peso, acha-se a falsa com n pesagens.

Passo. Seja M um conjunto qualquer de 2^{n+1} moedas com exatamente uma falsa (peso q) e as demais de peso $p > q$. Como $2^{n+1} = 2 \cdot 2^n$, divide-se M em dois conjuntos disjuntos M_1 e M_2 com 2^n moedas cada, e põe-se um em cada prato (1 pesagem). A falsa está em exatamente um dos dois; chame-o de M_i e o outro de M_j. Então

$$\text{peso}(M_i) = (2^n - 1)\cdot p + q \quad \text{e} \quad \text{peso}(M_j) = 2^n \cdot p, \quad \text{logo} \quad \text{peso}(M_j) - \text{peso}(M_i) = p - q > 0.$$

A balança não equilibra e o prato mais leve é o de M_i: a pesagem revela qual grupo contém a falsa. M_i é um conjunto de 2^n moedas com exatamente uma falsa, de peso q, e as demais de peso $p > q$ — exatamente a situação de *P(n)*. Pela hipótese de indução, *aplicada ao conjunto* M_i, a falsa de M_i (que é a falsa de M) é identificada com n pesagens. Total: $1 + n = n + 1$ pesagens. Logo *P(n+1)*.

Conclusão. Pelo princípio da indução (base *P(1)*, passo $P(n) \to P(n+1)$ para todo $n \geq 1$), *P(n)* vale para todo $n \geq 1$: com 2^n moedas, acha-se a falsa com n pesagens. ∎ (Se $0 \in \mathbb{N}$, a base *P(0)* com o mesmo passo, aplicado também em $n = 0$ — divide 2 moedas em grupos de 1, pesa, e *P(0)* identifica a falsa no grupo mais leve —, dá o resultado para todo $n \geq 0$.)

> **Fencepost.** Passo com $n = 1$ (o menor em que é aplicado, na leitura $n \geq 1$): 4 moedas → dois grupos de 2 → 1 pesagem → grupo mais leve de 2 moedas → base: 1 pesagem. Total $2 = n + 1$. O número de pesagens satisfaz $W(1) = 1$, $W(n+1) = W(n) + 1$, logo $W(n) = n = lg(2^n)$, o logaritmo do número de moedas.

3. Observações

Neste procedimento o resultado "equilíbrio" nunca ocorre: os dois pratos têm o mesmo número de moedas e só um contém a falsa.

n pesagens bastam, mas não são o mínimo para n ≥ 3. Cada pesagem tem 3 resultados possíveis, e há 2^n possibilidades para a posição da falsa; logo qualquer estratégia precisa de pelo menos $\lceil \log_3 2^n \rceil$ pesagens, e dividir em três grupos aproveita o resultado "equilíbrio". Exemplo: 8 moedas ($n = 3$) em 2 pesagens — pesa-se 3 contra 3; se equilibrar, a falsa está entre as 2 restantes, e pesa-se uma contra a outra; se não, está entre as 3 do prato mais leve, e pesa-se uma contra outra dessas três: o prato mais leve indica a falsa e, no equilíbrio, a falsa é a terceira. Para todo $n \geq 3$ isso dá uma estratégia com menos de n pesagens: faz-se a bissecção por $n - 3$ pesagens, até restar um grupo de $2^3 = 8$ moedas que contém a falsa, e resolve-se esse grupo em 2 pesagens como acima; total $(n - 3) + 2 = n - 1 < n$. O enunciado pede apenas que n pesagens sejam suficientes, que é o que a indução mostra.

Questão 4

Enunciado

Prove as assertivas abaixo usando o método da indução matemática.

a. $P(n) : n! > 2^n$, $\forall n \geq 4$

b. $P(n) : 2 + 4 + 6 + \ldots + 2 * n = n^2 + n$, $\forall n \geq 1$

c. $P(n) : 1 + 2 + 3 + \ldots + n = n * (n + 1)/2$, $\forall n \geq 1$

d. $T(n) = \{ 1 \text{ se } n = 1; \quad T(n - 1) + n \text{ se } n \geq 2 \}$
   A fórmula fechada para essa recorrência é $T(n) = n * (n + 1)/2$

e. $T(n) = \{ 1 \text{ se } n = 1; \quad 2 * T(n - 1) + 1 \text{ se } n \geq 2 \}$
   A fórmula fechada para essa recorrência é $T(n) = 2^n - 1$

(a) n! > 2^n, para todo n ≥ 4

P(n): $n! > 2^n$.

Base (n = 4). $4! = 24$ e $2^4 = 16$. Como $24 > 16$, *P(4)* é verdadeira.

Hipótese indutiva. Suponha $n! > 2^n$ para algum $n \geq 4$.

Passo. Queremos $(n+1)! > 2^{n+1}$. Da definição de fatorial,

$$(n+1)! = (n+1)\cdot n! > (n+1)\cdot 2^n \quad \leftarrow \text{aqui a hipótese é aplicada}$$

Como $n \geq 4$, tem-se $n + 1 \geq 5 > 2$, logo

$$(n+1)\cdot 2^n > 2\cdot 2^n = 2^{n+1}.$$

Encadeando, $(n+1)! > 2^{n+1}$, isto é, *P(n+1)*. ∎

> **A base em 4 é justa, não folgada.** A afirmação é *falsa* antes disso: $1! = 1 > 2$ é falso; $2! = 2 > 4$ é falso; $3! = 6 > 8$ é falso. Só em $n = 4$ ela passa a valer. Começar a base em $n = 1$ não seria apenas erro de forma — seria "provar" algo falso. E note que a restrição $n \geq 4$ é usada exatamente uma vez no passo: para garantir $n + 1 > 2$.

(b) $2 + 4 + 6 + \ldots + 2n = n^2 + n$, para todo $n \geq 1$

$P(n)$: $\Sigma_{i=1..n}\, 2i = n^2 + n$.

> **Base ($n = 1$).** Esquerda: $2 \cdot 1 = 2$. Direita: $1^2 + 1 = 2$. Iguais.
>
> **Hipótese indutiva.** Suponha $2 + 4 + \ldots + 2n = n^2 + n$ para algum $n \geq 1$.
>
> **Passo.** A soma até $2(n+1)$ é
>
> $$(2 + 4 + \ldots + 2n) + 2(n+1) = (n^2 + n) + 2n + 2 \quad \leftarrow \text{hipótese aplicada}$$
> $$= n^2 + 3n + 2.$$
>
> E a fórmula avaliada em $n+1$ dá
>
> $$(n+1)^2 + (n+1) = n^2 + 2n + 1 + n + 1 = n^2 + 3n + 2.$$
>
> As duas expressões coincidem, logo $P(n+1)$. ∎

(c) $1 + 2 + 3 + \ldots + n = n(n+1)/2$, para todo $n \geq 1$

$P(n)$: $\Sigma_{i=1..n}\, i = n(n+1)/2$.

> **Base ($n = 1$).** Esquerda: 1. Direita: $1 \cdot 2/2 = 1$. Iguais.
>
> **Hipótese indutiva.** Suponha $1 + 2 + \ldots + n = n(n+1)/2$ para algum $n \geq 1$.
>
> **Passo.**
>
> $$(1 + 2 + \ldots + n) + (n+1) = n(n+1)/2 + (n+1) \quad \leftarrow \text{hipótese aplicada}$$
>
> Fatorando $(n+1)$:
>
> $$= (n+1)\cdot(n/2 + 1) = (n+1)\cdot(n+2)/2,$$
>
> que é a fórmula avaliada em $n+1$. ∎

(d) $T(n) = T(n-1) + n$, $T(1) = 1 \implies T(n) = n(n+1)/2$

$P(n)$: $T(n) = n(n+1)/2$.

> **Base ($n = 1$).** Pela definição, $T(1) = 1$. Pela fórmula, $1 \cdot (1+1)/2 = 1$. Iguais.
>
> **Hipótese indutiva.** Suponha $T(n-1) = (n-1)\cdot n/2$ para algum $n \geq 2$.
>
> **Passo.** Como $n \geq 2$, vale o caso recursivo da definição — e é *dele* que se parte:
>
> $$T(n) = T(n-1) + n = (n-1)\cdot n/2 + n \quad \leftarrow \text{hipótese aplicada}$$
> $$= (n^2 - n)/2 + 2n/2 = (n^2 + n)/2 = n(n+1)/2.$$ ∎

> Primeiros valores pela recorrência: 1, 3, 6, 10, 15, 21, 28, 36 — que coincidem com $n(n+1)/2$.

(e) $T(n) = 2\cdot T(n-1) + 1$, $T(1) = 1 \implies T(n) = 2^n - 1$

$P(n)$: $T(n) = 2^n - 1$.

> **Base ($n = 1$).** Pela definição, $T(1) = 1$. Pela fórmula, $2^1 - 1 = 1$. Iguais.
>
> **Hipótese indutiva.** Suponha $T(n-1) = 2^{n-1} - 1$ para algum $n \geq 2$.
>
> **Passo.** Como $n \geq 2$, parte-se do caso recursivo da definição:
>
> $$T(n) = 2\cdot T(n-1) + 1 = 2\cdot(2^{n-1} - 1) + 1 \quad \leftarrow \text{hipótese aplicada}$$
> $$= 2^n - 2 + 1 = 2^n - 1.$$ ∎

> O fecho da conta é o $-2 + 1 = -1$: é ali que a fórmula se encaixa, e é o passo que não se deve pular. Primeiros valores pela recorrência: 1, 3, 7, 15, 31, 63, 127, 255 — que coincidem com $2^n - 1$.

> **Sobre a estrutura dos itens (d) e (e).** Nos dois, o passo começa aplicando a *definição recursiva* e termina na fórmula fechada. O caminho inverso — escrever a fórmula em n e manipulá-la até chegar em algo verdadeiro — é circular: usaria a tese para provar a tese. A distinção é sutil na leitura e decisiva na correção.

Questão 5

> Enunciado
>
> Seja a sentença aberta em $\mathbb{N}$:
>
> $$P(n): \quad 1 + 2 + \cdots + n = n(n+1)/2 + 1,$$
>
> mostre que:
> (i) Qualquer que seja $n \in \mathbb{N}$, se $P(n)$ é verdadeira, então $P(n+1)$ é verdadeira.
> (ii) $P(n)$ não é verdadeira para nenhum valor de $n \in \mathbb{N}$.

O passo vale, a base não — e sem base a indução não conclui nada. Seja $S(n) = 1 + 2 + \cdots + n = \Sigma_{k=1..n}\, k$. O enunciado não diz se $0 \in \mathbb{N}$; a solução cobre as duas convenções. Para $n = 0$, $S(0)$ é a soma vazia, igual a 0. Assim, $P(n)$ é a afirmação $S(n) = n(n+1)/2 + 1$.

(i) Para todo n ∈ ℕ, P(n) → P(n+1)

Seja $n \in \mathbb{N}$ arbitrário e suponha $P(n)$, isto é, $S(n) = n(n+1)/2 + 1$. Como $S(n+1) = S(n) + (n+1)$ (separação do último termo; vale também para $n = 0$, pois $S(1) = 0 + 1$),

$$\begin{aligned} S(n+1) &= S(n) + (n+1) = n(n+1)/2 + 1 + (n+1) \quad \leftarrow \text{hipótese P(n)} \\ &= [n(n+1) + 2(n+1)]/2 + 1 = (n+1)(n+2)/2 + 1. \end{aligned}$$

Como $(n+1)((n+1)+1)/2 = (n+1)(n+2)/2$, isto é exatamente $P(n+1)$. Como n foi arbitrário, a implicação vale para todo $n \in \mathbb{N}$. ∎

> A prova de (i) é direta e usa apenas a hipótese $P(n)$ e a definição de S; não usa (ii) nem a fórmula verdadeira de $S(n)$.

(ii) P(n) é falsa para todo n ∈ ℕ

Lema. Para todo $n \in \mathbb{N}$, $S(n) = n(n+1)/2$.

> **Base.** $S(1) = 1 = 1 \cdot 2/2$; e, se $0 \in \mathbb{N}$, $S(0) = 0 = 0 \cdot 1/2$.
>
> **Hipótese.** Para um $n \in \mathbb{N}$ fixado, $S(n) = n(n+1)/2$.
>
> **Passo.** $S(n+1) = S(n) + (n+1) = n(n+1)/2 + (n+1) = (n+1)(n+2)/2$ ← hipótese aplicada. Logo o Lema vale para todo $n \in \mathbb{N}$. ∎

(É a mesma identidade da Questão 4(c) desta lista, aqui estendida a $n = 0$.)

Conclusão de (ii). Para qualquer $n \in \mathbb{N}$, pelo Lema, o lado esquerdo de $P(n)$ vale $S(n) = n(n+1)/2$, e o lado direito vale $n(n+1)/2 + 1 = S(n) + 1$. Como $S(n) \neq S(n) + 1$, $P(n)$ é falsa. Como n foi arbitrário, $P(n)$ não é verdadeira para nenhum $n \in \mathbb{N}$. ∎

> **Por que (ii) não se prova só olhando a base.** Verificar que $P(1)$ é falsa ($1 \neq 2$) mostra que a base falha, mas *não* mostra que $P(n)$ é falsa para todo n: uma propriedade pode falhar nos primeiros valores e valer depois (por exemplo, $n! > 2^n$ é falsa em $n = 1, 2, 3$ e verdadeira para todo $n \geq 4$ — Questão 4(a) desta lista). Por isso (ii) precisa de um argumento que valha para **todo** n, e não só da base. Aqui esse argumento é o Lema, que mostra que os dois lados diferem por exatamente 1 em todo n. Não é o único caminho: a conta de (i) é reversível — como $S(n+1) = S(n) + (n+1)$ e $(n+1)(n+2)/2 + 1 = [n(n+1)/2 + 1] + (n+1)$, vale $S(n+1) = (n+1)(n+2)/2 + 1$ se e somente se $S(n) = n(n+1)/2 + 1$ —, logo $P(n+1) \to P(n)$, isto é, $\neg P(n) \to \neg P(n+1)$; uma indução sobre $\neg P$, com base $\neg P(1)$ (ou $\neg P(0)$), também prova (ii). Nos dois caminhos há base **e** passo: a falha da base, sozinha, não basta.

O que a questão ensina

(i) e (ii) juntos mostram que o passo indutivo sozinho não prova nada: aqui $P(n) \to P(n+1)$ vale para todo n e, mesmo assim, $P(n)$ é falsa para todo n. O que falta é um ponto de partida: $P(1)$ (ou $P(0)$) é falsa e, por (ii), nenhum $P(m)$ é verdadeiro, de modo que a cadeia de implicações nunca é disparada, qualquer que seja o valor inicial escolhido. A questão mostra que a base é indispensável: um passo válido, sozinho, não prova nada.

Informações que podem ser úteis

> Anexo da lista (transcrição literal)
>
> **Somatórios e Logaritmos:**
>
> - $\Sigma_{i=1}^{n}\,(i) = n(n+1)/2$
> - $\Sigma_{i=1}^{n}\,(a^i) = a(a^n - 1)/(a - 1)$
> - $\log\,(a * b) = \log a + \log b$
> - $\log\,(a/b) = \log a - \log b$
> - $\log_b a = \log_c a / \log_c b$
> - $\log_b a^c = c * \log_b a$
> - $\lg a = \log_2 a$
>
> **Hierarquia de funções:**
>
> A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico:
>
> $$1 < \log(n) < \sqrt{n} < n < n * \log(n) < n^c < c^n < n! < n^n$$
>
> onde c é uma constante arbitrária inteira ≥ 2.

Atualização de 09/09/2026. As Questões 3 e 5 são novas; a Questão 4 é a antiga Questão 3, com o mesmo texto. Textos-fonte verificados: `geradores/dados/lista03/q3-moedas-limpo.md` e `q5-passo-sem-base-limpo.md`. Na Questão 3, 4 de 4 verificadores sem defeito; uma observação lateral ("não são o mínimo para n ≥ 3") só estava demonstrada para n = 3 e ganhou a construção geral, reverificada com 4 de 4. Na Questão 5, 3 de 4 verificadores derrubaram uma nota que dizia que (ii) "exige" o Lema — falso, há o caminho por ¬P —; corrigida, 4 de 4 na reverificação. A bissecção foi simulada para n = 0…12 em toda posição da moeda falsa, e a estratégia de n − 1 pesagens para n = 3…11; as identidades da Questão 5 em n = 0…10^5 (`verificacao/atualizacao-2026-09-chao-deterministico.py`).
Sobre esta resolução. As fórmulas fechadas das recorrências (d) e (e) da Questão 4 foram conferidas por recursão direta em n = 1…2999 e n = 1…599 respectivamente; as somas de (b) e (c) em n = 1…4999; e a fronteira de (a) verificada item a item, confirmando que a afirmação é falsa em n = 1, 2, 3 e verdadeira a partir de n = 4. As constantes da Questão 1 foram calibradas e testadas em toda a faixa n = 1…200, incluindo o caso degenerado $\lg 1 = 0$. Na Questão 2, a ausência de cota inferior de ordem n^2 foi verificada para c = 1, 0,5, 0,01 e 0,0001. Os scripts estão em `resolucoes/verificacao/`. As demonstrações foram

submetidas a verificadores independentes em contexto isolado, que receberam apenas o enunciado e a prova final. Lista de estudo, sem entrega — IFMG Campus Formiga, 2026/2.